Relatório: Cálculo Numérico 2ª Unidade

Equipe: Arthur Lincoln, Raioni Felix e Hiago Lima. Código: github.com/Hiago-Lima/Problemas-Calculo-Numerico.

Problema	Tópico	Assunto	Método	Programa	Quem fez
1.2	1. Sistemas lineares, métodos diretos	Produção de três componentes elétricos (metal, plástico e borracha)	métodos diretos	`main.py` (Gauss, Gauss-Jordan e LU)	Raioni Felix
2.2	2. Sistemas lineares, métodos iterativos	Forças nas barras de uma treliça	Gauss-Seidel	`gauss_seidel.py`, `main.py`	Arthur Lincoln
3.1	3. Interpolação e mínimos quadrados	Lei de Moore, com log10 do número de transistores	Mínimos quadrados	`executar_problema_3_1.py`	Hiago Lima
4.1	4. Integração numérica	Área da seção reta de um rio	Trapézio e Simpson repetidos	`simpson.py`, `integracao.py`, `main.py`	os três

1. Problema 1.2 — Produção de componentes elétricos (Tópico 1)

O problema

O problema 1.2 é do Tópico 1 (sistemas lineares, métodos diretos). Um engenheiro supervisiona a produção de três tipos de componentes elétricos, que usam metal, plástico e borracha. A tabela do enunciado dá o consumo de cada material, em gramas, por componente produzido:

Componente	Metal (g)	Plástico (g)	Borracha (g)
1	15	0,30	1,0
2	17	0,40	1,2
3	19	0,55	1,5

Estão disponíveis por dia 3,89 kg de metal, 0,095 kg de plástico e 0,282 kg de borracha, e a pergunta é quantos componentes de cada tipo podem ser produzidos por dia. O enunciado divide o trabalho em três itens: (a) apresentar o problema como um sistema de equações lineares, (b) fazer um programa que resolva problemas semelhantes usando métodos diretos e (c) encontrar a solução com os dados do problema.

Montagem do sistema (item a)

Sejam x1, x2 e x3 as quantidades produzidas por dia de cada componente. Para cada material, o que é gasto na produção tem que ser igual ao que está disponível, o que dá uma equação por material: o consumo de cada componente vezes a quantidade produzida dele, somados, igual à disponibilidade. A tabela está em gramas e a disponibilidade em quilogramas, então passamos tudo para gramas (1 kg = 1000 g): 3,89 kg = 3890 g, 0,095 kg = 95 g e 0,282 kg = 282 g.

```
15x1 + 17x2 + 19x3   = 3890     (metal)
0,3x1 + 0,4x2 + 0,55x3 = 95     (plástico)
  x1 + 1,2x2 + 1,5x3  = 282     (borracha)
```

Em forma matricial, A x = b, cada linha de A é um material e cada coluna é um componente (a tabela do enunciado, transposta):

```
A = [ 15    17    19   ]     x = [ x1 ]     b = [ 3890 ]
    [ 0,3   0,4   0,55 ]         [ x2 ]         [ 95   ]
    [ 1     1,2   1,5  ]         [ x3 ]         [ 282  ]
```

São 3 equações e 3 incógnitas, e a matriz não tem zeros. O determinante de A é 0,04, diferente de zero, então o sistema tem solução única.

Métodos diretos e programa (item b)

Um método direto resolve o sistema com um número fixo de operações, sem chute inicial (ao contrário do Gauss-Seidel da seção 2). O programa tem três:

- **Eliminação de Gauss.** Usa operações entre linhas para deixar a matriz triangular superior (zeros abaixo da diagonal) e resolve de baixo para cima, por retrosubstituição. Usamos pivoteamento parcial: em cada coluna o pivô é o elemento de maior valor absoluto, da linha atual para baixo, e as linhas são trocadas quando preciso. Isso evita dividir por zero ou por um número muito pequeno e reduz erros de arredondamento. Se o melhor candidato a pivô é praticamente zero (até 10^-12 vezes o maior coeficiente), o sistema é singular e o método avisa.
- **Gauss-Jordan.** Continua a eliminação até transformar A na matriz identidade, e a coluna do lado direito vira a solução, sem retrosubstituição. Como zera a coluna também acima do pivô, faz mais contas: cerca de $n^3/2$ multiplicações, contra $n^3/3$ do Gauss.
- **Fatoração LU.** Decompõe P A = L U, em que L é triangular inferior com 1 na diagonal (guarda os multiplicadores da eliminação), U é triangular superior e P é a troca de linhas do pivoteamento. Depois resolve dois sistemas triangulares: L y = P b (substituição progressiva) e U x = y (substituição retroativa). A fatoração depende só de A, então poderia ser reaproveitada para vários vetores b.

O programa. Cada método está em um arquivo (`metodo_diretoGaus.py`, `metodo_jordan.py` e `metodo_lu.py`), e a função `sistemaEquacaoLinear(A, b, metodo, passos)`, do arquivo `sistemaEquacaoLinear.py`, escolhe qual usar (`gauss`, `jordan` ou `lu`). Se uma lista for passada em `passos`, cada método anota nela as matrizes e contas de cada etapa, para mostrar o passo a passo. O menu está em `main.py`: resolve o problema 1.2 do enunciado, outro problema de produção (a pessoa digita a tabela de consumo e a disponibilidade) ou qualquer sistema linear digitado, por um dos métodos ou pelos três, comparando os resultados, e no fim pergunta se quer fazer outro cálculo ou sair. Sistemas singulares e entradas inválidas são recusados com mensagem.

```
python main.py                  # programa com menu
python teste_problema_1-2.py    # resolve o 1.2 pelos 3 métodos e compara o tempo
```

O arquivo `teste_problema_1-2.py` resolve o problema 1.2 pelos três métodos, mostra o passo a passo de cada um, confere as soluções por substituição e compara o tempo de execução.

Resultados (item c)

Os três métodos dão a mesma solução:

Componente	Quantidade por dia
1	90
2	60
3	80

Conferência por substituição nas três equações:

```
metal:     15·90 + 17·60 + 19·80       = 1350 + 1020 + 1520 = 3890 g
plástico:  0,3·90 + 0,4·60 + 0,55·80   =   27 +   24 +   44 =   95 g
borracha:  1·90 + 1,2·60 + 1,5·80      =   90 +   72 +  120 =  282 g
```

O maior resíduo |A x - b| é da ordem de 10^-13 (4,55 × 10^-13 no Gauss e na LU, 0 no Gauss-Jordan), e a maior diferença entre os métodos é 4,19 × 10^-13, que é só arredondamento. No Gauss, o pivô da primeira coluna é 15 e, na segunda, o programa troca as linhas 2 e 3, porque o pivô passa de 0,06 para 0,0667, o maior da coluna.

O teste também compara o tempo de cada método. Em uma execução (os valores mudam de uma máquina para outra), tomando o mais rápido como 1,00:

Sistema	Gauss	Gauss-Jordan	LU
Problema 1.2 (3 × 3)	1,00	1,07	1,30
60 × 60, gerado ao acaso	1,00	1,48	1,00

Análise

A solução faz sentido. Os valores 90, 60 e 80 são inteiros e não negativos, e a conferência mostra que a produção usa exatamente todo o material disponível (3890, 95 e 282 g).

Qual método usar. No sistema 3 × 3 as diferenças são de poucos microssegundos e incluem o custo fixo do próprio Python, então qualquer um dos três serve. No sistema 60 × 60 o custo da eliminação domina: Gauss e LU empatam e o Gauss-Jordan fica cerca de 48% mais lento, o que combina com $n^3/2$ contra $n^3/3$ multiplicações. Para este tipo de fábrica, em que a tabela de consumo é fixa e só a disponibilidade de material muda, a LU é a mais indicada: dá para fatorar A uma vez e, para cada novo b, resolver só os dois sistemas triangulares. O programa não aproveita isso, porque refaz a fatoração a cada chamada.

Limites. O programa resolve sistemas quadrados com solução única. Ele não confere se as quantidades saem negativas ou não inteiras: com outros dados, a solução pode estar certa como sistema e não fazer sentido como produção (por exemplo, -5 componentes), e essa interpretação fica com quem usa.

2. Problema 2.2 — Treliça (Tópico 2, Gauss-Seidel)

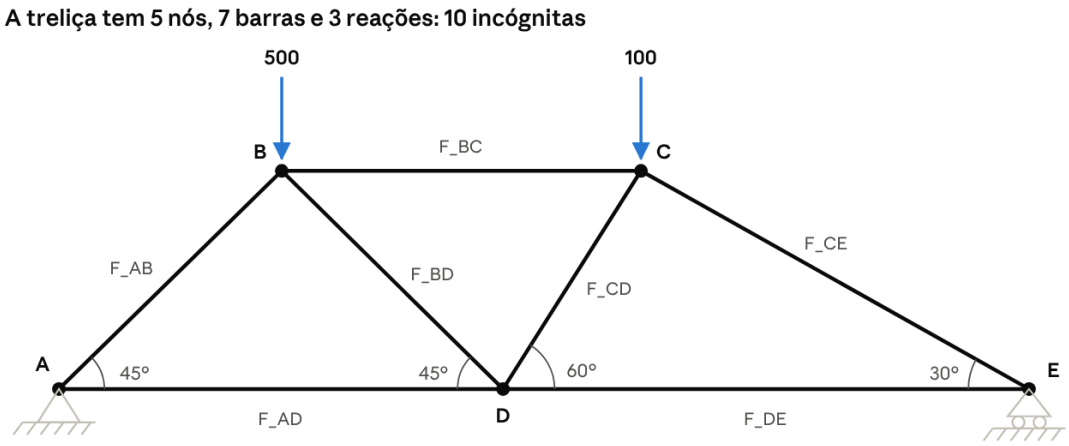
2.1 O problema

O problema 2.2 pede as forças nas barras de uma treliça plana de 5 nós e 7 barras, sob duas cargas verticais, resolvendo o sistema linear que resulta do equilíbrio dos nós pelo método de Gauss-Seidel, com precisão 0,0001. O enunciado divide o trabalho em três itens: (A) escrever o sistema linear da treliça, (B) fazer um programa de Gauss-Seidel que resolva esse sistema e outros parecidos, e (C) encontrar a solução com os dados da figura.

A treliça é estaticamente determinada e tem juntas articuladas, então em cada nó a soma das forças horizontais e a soma das forças verticais são iguais a zero. Lemos da figura os seguintes dados:

- **Nós.** A é o apoio fixo, à esquerda. B e C ficam no alto, B à esquerda e C à direita. D fica embaixo, no meio. E é o apoio móvel, à direita. Essas letras são nossas, a figura não nomeia os nós.
- **Cargas.** 500 no nó B e 100 no nó C, as duas verticais e para baixo. O enunciado não dá a unidade, então as forças calculadas saem na mesma unidade das cargas.
- **Barras.** AB, BC, BD, CD, CE, AD e DE, com os ângulos da tabela abaixo. A barra BC é horizontal, assim como AD e DE.

Barra	Liga	Ângulo com a horizontal
F_AB	A e B	45°
F_BC	B e C	0°
F_BD	B e D	45°
F_CD	C e D	60°
F_CE	C e E	30°
F_AD	A e D	0°
F_DE	D e E	0°



Cargas verticais para baixo, na unidade do enunciado. Tração positiva, compressão negativa.
Apoio fixo em A (reações Ax e Ay) e apoio móvel em E (reação Ey).

treliça do problema 2.2 · 5 nós, 7 barras, 2 cargas

Os ângulos e as cargas são os da figura do enunciado; as letras dos nós e os nomes das barras são os usados no resto do texto.

Adotamos barra tracionada como positiva (ela puxa os nós das pontas) e barra comprimida como negativa. O apoio fixo A tem reação horizontal Ax e vertical Ay, positivas para a direita e para cima. O apoio móvel E só tem reação vertical Ey. São 10 incógnitas (7 forças e 3 reações) e 10 equações (2 por nó).

2.2 Montagem do sistema linear (item A)

Cada nó gera duas equações, a soma das forças em x e a soma das forças em y, e o sistema final é 10 × 10. Uma barra tracionada puxa o nó na direção do outro extremo dela, então a componente de cada força no nó usa o seno ou o cosseno do ângulo da barra, com o sinal dessa direção. No nó B, por exemplo, a barra AB puxa para baixo e para a esquerda, a BD para baixo e para a direita e a BC para a direita.

As equações de cada nó, com as cargas de 500 e 100 já colocadas:

```
Nó A   soma Fx:   F_AB cos45 + F_AD + Ax = 0
        soma Fy:   F_AB sen45 + Ay = 0

Nó B   soma Fx:  -F_AB cos45 + F_BD cos45 + F_BC = 0
        soma Fy:  -F_AB sen45 - F_BD sen45 - 500 = 0

Nó C   soma Fx:  -F_BC - F_CD cos60 + F_CE cos30 = 0
        soma Fy:  -F_CD sen60 - F_CE sen30 - 100 = 0

Nó D   soma Fx:  -F_AD - F_BD cos45 + F_CD cos60 + F_DE = 0
        soma Fy:   F_BD sen45 + F_CD sen60 = 0

Nó E   soma Fx:  -F_CE cos30 - F_DE = 0
        soma Fy:   F_CE sen30 + Ey = 0
```

Passando as cargas para o lado direito e calculando os senos e cossenos, o sistema fica assim. É o mesmo que o programa imprime na tela:

```
Nó A, soma Fx : 0.7071*F_AB + F_AD + Ax = 0
Nó A, soma Fy : 0.7071*F_AB + Ay = 0
Nó B, soma Fx : -0.7071*F_AB + F_BC + 0.7071*F_BD = 0
Nó B, soma Fy : -0.7071*F_AB - 0.7071*F_BD = 500
Nó C, soma Fx : -F_BC - 0.5*F_CD + 0.866*F_CE = 0
Nó C, soma Fy : -0.866*F_CD - 0.5*F_CE = 100
Nó D, soma Fx : -0.7071*F_BD + 0.5*F_CD - F_AD + F_DE = 0
Nó D, soma Fy : 0.7071*F_BD + 0.866*F_CD = 0
Nó E, soma Fx : -0.866*F_CE - F_DE = 0
Nó E, soma Fy : 0.5*F_CE + Ey = 0
```

A matriz é esparsa: dos 100 coeficientes, 25 são diferentes de zero. É por isso que o enunciado sugere um método iterativo.

2.3 Método e programa (item B)

O Gauss-Seidel isola a incógnita x_i na equação i e usa, na mesma iteração, os valores que já foram atualizados:

```
x_i(k+1) = ( b_i - soma[j < i] a_ij * x_j(k+1) - soma[j > i] a_ij * x_j(k) ) / a_ii
```

O critério de parada é o do slide do professor, a diferença relativa entre duas iterações:

```
dR = max |x_i(k+1) - x_i(k)| / max |x_i(k+1)|     (para quando dR < 0,0001)
```

Zeros na diagonal. Na ordem natural das incógnitas o sistema não pode ser resolvido por Gauss-Seidel, porque há zeros na diagonal (a equação do nó A em y, por exemplo, não tem F_BC). O programa troca a ordem das incógnitas: procura as ordens em que nenhum elemento da diagonal é zero e fica com a de menor raio espectral (explicado na seção 2.5). Na treliça, só existem quatro ordens assim, e a escolhida põe na diagonal:

Equação	A Fx	A Fy	B Fx	B Fy	C Fx	C Fy	D Fx	D Fy	E Fx	E Fy
Incógnita na diagonal	Ax	Ay	F_BC	F_AB	F_CE	F_CD	F_AD	F_BD	F_DE	Ey

O programa. O método e a montagem do sistema da treliça estão em `gauss_seidel.py`, e o programa com menu está em `main.py`. Ele resolve qualquer sistema A x = b, não só o da treliça. O menu tem duas opções: a treliça do enunciado, em que as cargas e os ângulos podem ser trocados (Enter mantém os valores do enunciado), e a digitação de outro sistema, com uma equação por linha (coeficientes e termo independente separados por espaço, aceitando vírgula ou ponto). Em seguida o programa pergunta a precisão, o máximo de iterações e o chute inicial (zeros, b_i / a_ii ou digitado). Ele mostra o sistema, os critérios de convergência, a tabela de iterações, a solução e uma verificação por substituição, e no fim pergunta se o usuário quer fazer outro cálculo ou sair. Sistemas singulares e entradas inválidas são recusados com mensagem.

Para executar (precisa de Python 3 e numpy):

```
python main.py                    # programa
python teste_gauss_seidel.py      # testes
```

Os testes conferem o exemplo do slide 27 do professor, que dá x = 1,002, y = 0,998 e z = -1 em 4 iterações, e comparam a solução da treliça com a solução direta.

2.4 Resultados (item C)

Com os dados da figura, chute inicial zero e dR < 0,0001, o Gauss-Seidel converge em 40 iterações. A tabela compara o resultado com a solução direta do mesmo sistema (eliminação de Gauss feita pelo numpy), usada só para conferir.

Tabela 2.1. Solução da treliça (cargas 500 em B e 100 em C).

Incógnita	Gauss-Seidel	Solução direta	Situação
F_AB	-599,8516	-599,8627	compressão
F_BC	-348,3311	-348,3340	compressão
F_BD	-107,2381	-107,2441	compressão
F_CD	87,5595	87,5644	tração
F_CE	-351,6576	-351,6660	compressão
F_AD	424,1859	424,1670	tração
F_DE	304,5444	304,5517	tração
Ax	0,0140	0,0000	sem reação horizontal
Ay	424,1719	424,1670	para cima
Ey	175,8288	175,8330	para cima

A maior diferença para a solução direta é 0,0189, em F_AD, e o maior resíduo |A x - b| é 0,041. Com chute inicial b_i / a_ii o método converge em 36 iterações, quase o mesmo.

Tabela 2.2. Evolução do critério de parada.

Iteração k	1	2	5	10	20	30	40
dR	1,0	0,895	1,596	0,2235	0,0143	0,00083	0,000047

O resultado passa em três conferências simples. As reações somam 424,17 + 175,83 = 600, que é a carga total. Não há carga horizontal, então Ax é praticamente zero. E o equilíbrio vertical do nó B fecha: (599,85 + 107,24) × sen 45° = 500,0. A corda de cima (BC) e as diagonais AB, BD e CE ficam comprimidas, e a corda de baixo (AD e DE) e a diagonal CD ficam tracionadas.

2.5 Análise

Os critérios dos slides não garantem a convergência, mas o método converge. Na ordem escolhida, o critério das linhas dá máximo 2,21 e o de Sassenfeld dá M = 2,35, os dois acima de 1. Não é questão de ordem: a equação do nó A em x tem coeficientes 0,7071, 1 e 1, então, qualquer que seja a incógnita posta na diagonal, a soma dos outros dois passa de 1,7 vezes o elemento da diagonal, e o critério das

linhas falha. Para o de Sassenfeld, testamos as quatro ordens possíveis de diagonal em todas as ordens de varredura, e nenhuma combinação dá M menor que 1.

Os dois critérios são condições suficientes, não necessárias, e falhar neles não prova que o método diverge. A condição que decide é o raio espectral da matriz de iteração do Gauss-Seidel: o método converge, para qualquer chute inicial, se e somente se ele for menor que 1. Na ordem escolhida ele vale 0,7598, então o erro cai em média a cada iteração por um fator de cerca de 0,76, e depois de 40 iterações isso dá 0,76^40 ≈ 1,7 × 10^-5. O programa calcula e mostra os três números (linhas, Sassenfeld e raio espectral), e usa o raio espectral para escolher a ordem das incógnitas.

A convergência é lenta e irregular no começo. Com chute zero, o dR passa de 1 nas iterações 4 e 5 (Tabela 2.2) e só fica abaixo de 0,1 na iteração 13. Ele também não diminui sempre: depois de cair a 0,0023 na iteração 27, volta a subir para 0,0033 na 28.

Precisão 0,0001 é do dR, não das casas decimais. O critério de parada controla a diferença relativa entre duas iterações, e a diferença para a solução exata é maior que isso: 0,019 em F_AD, que vale 424, ou seja, um erro relativo de cerca de 4,5 × 10^-5 nessa barra. Nas forças desta treliça, só a parte inteira e, com alguma folga, a primeira decimal são confiáveis. Para ter mais casas é só pedir uma precisão menor ao programa.

Limites. O sistema desta treliça tem só 10 incógnitas, e a solução direta é instantânea e exata. O método iterativo compensa em treliças com centenas de nós, em que a matriz é esparsa e grande. O programa serve para a treliça do enunciado com outras cargas e outros ângulos, e para qualquer outro sistema linear digitado (opção 2 do menu). Outra treliça precisa ter o sistema montado à mão, como na seção 2.2.

3. Problema 3.1 — Lei de Moore (Tópico 3)

O problema

O problema 3.1 trata da Lei de Moore, que afirma que a quantidade de transistores em circuitos integrados dobra a cada 18 meses. A partir de uma tabela com 11 chips lançados entre 1971 (Intel 4004, com 2.250 transistores) e 2000 (Pentium 4, com 42.000.000 de transistores), pede-se propor uma função N = f(a) para prever o número de transistores de chips lançados em 2010 e em 2020. O enunciado orienta transformar previamente o eixo dos transistores aplicando log10(N).

Modelagem e linearização

O crescimento exponencial é modelado por N(a) = C · 10^(a1 · a). Tomando o logaritmo na base 10 dos dois lados, obtemos uma relação linear:

```
y = log10(N) = a0 + a1 * a,    com a0 = log10(C)
```

Com as coordenadas (a_i, y_i), em que y_i = log10(N_i), aplicamos o Método dos Mínimos Quadrados (MMQ), minimizando a soma dos resíduos quadráticos. O sistema de equações normais 2x2 resultante é:

```
[ n      Σa_i   ] [ a0 ]   [ Σy_i       ]
[ Σa_i   Σa_i²  ] [ a1 ] = [ Σ(a_i*y_i) ]
```

Para os 11 chips da tabela (n = 11): Σa = 21.841, Σa² = 43.367.485, Σy ≈ 59,724260 e Σ(a*y) ≈ 118.748,032246.

Resultados e previsões

Resolvendo o sistema pelos somatórios:

```
a1 = (n*Σ(a*y) - Σa*Σy) / (n*Σa² - (Σa)²)  ≈  0,137183
a0 = (Σy - a1*Σa) / n                      ≈  -266,953401
```

A equação linearizada obtida é log10(N) = -266,953401 + 0,137183 · a, que corresponde à função exponencial de previsão N(a) = 10^(-266,953401 + 0,137183 · a). Aplicando aos anos pedidos no enunciado:

Ano	log10(N)	N previsto
2010	8,784224	6,08 × 10^8 (608.448.217)
2020	10,156053	1,43 × 10^10 (14.323.613.778)

O coeficiente de determinação do ajuste linear nos logaritmos é R^2 = 98,95% (erro quadrático total de 0,2365), o que indica boa aderência aos dados históricos.

Programas

O código está em módulos na pasta Código/Problema 3.1: `minimos_quadrados.py` traz o cálculo analítico do MMQ e as métricas de erro; `teste_problema_3_1.py` traz os testes automatizados; e `executar_problema_3_1.py` é a interface com menu interativo, para novas entradas ou consultas de anos. Há também uma versão com SciPy (`minimos_quadrados_scipy.py`), que usa `scipy.optimize.curve_fit` para obter a matriz de covariância das incertezas (a0 = -266,95 ± 9,34 e a1 = 0,1372 ± 0,0047) e um benchmark comparativo.

4. Problema 4.1 — Seção reta de um rio (Tópico 4)

O problema

A determinação da área da seção reta de rios e lagos é importante em projetos de prevenção de enchentes (para o cálculo da vazão da água) e em projetos de reservatórios (para o cálculo do volume de água). Sem sonar, o engenheiro trabalha com profundidades medidas em pontos discretos da superfície. O problema 4.1 pede uma aplicação que calcule a área da seção reta do rio da figura do enunciado por integração numérica, com as regras do Trapézio e de Simpson repetidas, considerando pontos igualmente espaçados.

Lemos da figura as profundidades, em metros, a cada 2 m a partir da margem esquerda (h = 2 m, 10 intervalos):

x (m)	0	2	4	6	8	10	12	14	16	18	20
Profundidade (m)	0	1,8	2	4	4	6	4	3,6	3,4	2,8	0

Como só temos medidas pontuais, não existe um valor exato para comparar. A área é aproximada somando pedaços sob a curva do fundo, e os dois métodos diferem na forma de aproximar cada pedaço: o trapézio liga os pontos por retas e o Simpson usa parábolas.

Trapézio repetido

O trapézio repetido dá 63,2 m² para a área da seção reta do rio, com as profundidades da figura e 10 intervalos de 2 m.

Método. Em cada intervalo entre dois pontos vizinhos, a regra troca a curva por uma reta e calcula a área do trapézio formado, de altura h e bases y_i e y_(i+1): A_i = h/2 · (y_i + y_(i+1)). Somando os n trapézios, cada ponto interno aparece em dois deles (como base direita de um e base esquerda do seguinte) e por isso pesa 2; só y0 e yn aparecem uma vez:

```
I ≈ (h/2) * [ y0 + 2*(y1 + y2 + ... + y(n-1)) + yn ]
```

A regra é exata para retas (polinômios de grau até 1) e não exige número par de intervalos. Quando a função é suave, o erro total é da ordem de h², e o limite do erro é (b - a)/12 * h² * máx|f''|.

Aplicação ao rio. Com as profundidades da tabela da subseção anterior (h = 2 m, n = 10):

```
y1 + y2 + ... + y9 = 1,8 + 2 + 4 + 4 + 6 + 4 + 3,6 + 3,4 + 2,8 = 31,6

I = (2/2) * [ 0 + 2*31,6 + 0 ] = 63,2 m²
```

Programa. O arquivo `integracao.py` tem a função `trapezio_repetido(y, h)`, que recebe a lista de valores y0 a yn e o passo h e devolve a integral. A função recusa, com mensagem, menos de 2 pontos, h que não seja positivo e valores que não sejam números finitos.

```
python teste_integracao.py
```

Os testes conferem o resultado do rio (63,2), a exatidão para retas, o erro de x² entre 0 e 1 com n = 2 (dá 0,375 em vez de 1/3, e a diferença de 1/24 é a que a fórmula do erro prevê) e a integral de sen(x) entre 0 e π, que vale 2. Nela o erro é 0,104 com n = 4 e 0,026 com n = 8, cerca de 4 vezes menor ao dobrar n, o que combina com a ordem h². Também conferem que números ímpares de intervalos são aceitos e que entradas inválidas são recusadas.

Simpson 1/3 repetido

O Simpson 1/3 repetido dá 66,4 m² para a área da seção reta do rio, com as profundidades da figura e 10 intervalos de 2 m.

Método. Em cada par de intervalos vizinhos, a regra troca a função por uma parábola que passa por três pontos consecutivos e integra a parábola. Somando todos os pares, com passo h e n intervalos:

```
I ≈ (h/3) * [ y0 + 4*(y1 + y3 + ... + y(n-1)) + 2*(y2 + y4 + ... + y(n-2)) + yn ]
```

Os pontos de índice ímpar pesam 4, os de índice par (menos as pontas) pesam 2 e as pontas pesam 1. Cada parábola usa dois intervalos, então n tem que ser par. A regra é exata para polinômios de grau até 3. Quando a função é suave, o erro total é da ordem de h^4, contra h^2 no trapézio repetido; o limite do erro é (b - a)/180 * h^4 * máx|f''''|.

Aplicação ao rio. As profundidades lidas da figura, em metros, a cada 2 m a partir da margem esquerda (h = 2 m, n = 10, que é par):

x (m)	0	2	4	6	8	10	12	14	16	18	20
Profundidade (m)	0	1,8	2	4	4	6	4	3,6	3,4	2,8	0

```
Peso 4 (x = 2, 6, 10, 14, 18):  1,8 + 4 + 6 + 3,6 + 2,8 = 18,2
Peso 2 (x = 4, 8, 12, 16):      2 + 4 + 4 + 3,4          = 13,4

I = (2/3) * [ 0 + 4*18,2 + 2*13,4 + 0 ] = (2/3) * 99,6 = 66,4 m²
```

As profundidades são medidas pontuais, então não existe um valor exato para comparar. A conferência do método está nos testes abaixo.

Programa. O arquivo `simpson.py` tem a função `simpson_repetido(y, h)`, que recebe a lista de valores y0 a yn e o passo h e devolve a integral. O programa principal a usa com `from simpson import simpson_repetido`. A função recusa, com mensagem, número ímpar de intervalos, menos de 2 intervalos, h que não seja positivo e valores que não sejam números finitos. Com 3 intervalos, por exemplo, a mensagem é "A regra exige número par de intervalos, há 3 (4 pontos) no momento." Como a regra pede n par, uma tabela com número ímpar de intervalos (7, por exemplo) não pode ser integrada só por Simpson 1/3. Uma saída seria usar Simpson 3/8 em três dos intervalos, o que não foi implementado.

```
python teste_simpson.py
```

Os testes conferem o resultado do rio (66,4), a exatidão para x^2 e x^3 e a integral de sen(x) entre 0 e π, que vale 2. Nela o erro é 0,0046 com n = 4 e 0,00027 com n = 8, cerca de 17 vezes menor ao dobrar n, o que combina com a ordem h^4.

Comparação dos dois métodos

O Trapézio repetido resulta em 63,20 m² e o Simpson 1/3 repetido em 66,40 m², uma diferença de 3,20 m² (4,82% do valor de Simpson). O Simpson tem erro de ordem $O(h^4)$, contra $O(h^2)$ do Trapézio, e por isso costuma ser mais preciso quando o fundo do rio é suave: o arco de parábola acompanha a curvatura melhor que os segmentos retos. Como só temos as medidas e não a área verdadeira, não dá para dizer qual dos dois valores está mais perto dela; o Trapézio ter dado menos que o Simpson é coerente com um perfil de fundo arredondado, mas a diferença é só um indicativo.

Programa principal (`main.py`). Para juntar os métodos, há uma interface interativa em `main.py`, na pasta do problema 4.1. Ela conecta os módulos `integracao.py` (Raioni Felix) e `simpson.py` (Arthur Lincoln), exibe a tabela dos pontos (x, y) e mostra a comparação lado a lado entre os métodos. Permite calcular com os dados do rio do enunciado ou inserir novos pontos com qualquer passo h constante, em laço, com opção de fazer novo cálculo ou sair. O problema também está no menu geral da raiz do repositório (`python main.py`).

5. Participação de cada membro

Arthur Lincoln: ficou com o problema 2.2 (o programa de Gauss-Seidel, o sistema da treliça e a seção 2 deste relatório), e no problema 4.1, dividido entre os três, fez o Simpson repetido (o arquivo simpson.py e a subseção sobre Simpson da seção 4 deste relatório).

Raioni Felix: ficou com o problema 1.2 (os métodos de Gauss, Gauss-Jordan e LU, o seletor sistemaEquacaoLinear, o programa com menu main.py e o teste do problema 1.2) e, no problema 4.1, dividido entre os três, fez o trapézio repetido (o arquivo integracao.py e o teste dele).

Hiago Lima: ficou com o problema 3.1 (modelagem da Lei de Moore por mínimos quadrados, programa analítico, testes automatizados e versão com SciPy) e, no problema 4.1, fez a interface interativa (main.py) que reúne o trapézio e o Simpson com o menu e a comparação dos métodos.